Software Engineering Project · AI-Powered Co-Working Space Asset Manager

Hooria's Step-by-Step Build Guide

Notifications, Usage History and the two AI screens, with the full code · partner: Misbah

Read this first

This guide walks you through your whole part, one file at a time. Every file has its **complete code**, an explanation of the important lines, and a short test so you know it works before you move on. All of this code was run and tested on the real app (MySQL + Chrome) with everyone else's finished modules.

How to use it: do the steps in order. For each one, open the file, replace what is inside with the code shown, read the "What the code does" notes, then do the "Test it" part. Don't skip the tests: if step 1 is broken, step 3 will look broken too. Typing the code yourself helps you explain it in the viva.

What you will build

#	What	File	SRS
1	Notifications API (4 endpoints)	`server/src/modules/notifications/routes.js`	FR-NT-01, 03
2	"Due tomorrow" reminder job	`server/src/modules/notifications/jobs.js`	FR-NT-01
3	Bell with unread count	`client/src/modules/notifications/NotificationBell.jsx`	FR-NT-01
4	Notifications page	`client/src/modules/notifications/NotificationsPage.jsx`	FR-NT-01 to 03
5	History API (2 endpoints)	`server/src/modules/history/routes.js`	FR-UH-01 to 03
6	My History page	`client/src/modules/history/MyHistoryPage.jsx`	FR-UH-02, 03
7	Asset History page (admin)	`client/src/modules/history/AssetHistoryPage.jsx`	FR-UH-01, 03
8	AI Recommendation screen	`client/src/modules/ai/RecommendPage.jsx`	FR-RC-01, 04 to 06
9	AI Assistant chat screen	`client/src/modules/ai/AssistantPage.jsx`	FR-AI-01, 07

You do **not** need to change `server/src/services/notify.js` : it already works, and Hamza's and Azlaan's code already calls it, so your notifications fill up as soon as people use the app. Misbah's AI server side (`/api/ai/...`) is finished too; your AI screens just call it.

Rules that keep the team's code from clashing

- [ ] Only change the 9 files above. Don't edit `app.js` , `routes.jsx` , `components/` , `styles.css` or other people's folders.
- [ ] Keep every URL and response shape exactly as in `docs/API.md` . Other screens depend on them.
- [ ] Screens use the shared pieces from `components/ui` (Neon Noir theme, see `docs/UI_TEMPLATE.md`). Never type hex colours or add `.css` files.
- [ ] Every list needs three states: loading (skeleton), empty (`EmptyState`) and error (`ErrorBox`).

0 Get the app running

You need **Node.js 20+**, **MySQL 8** (or XAMPP's MySQL) and **Git**. First get the latest code with everyone's finished modules, then make your own branch:

```
# in the project folder
git checkout main
git pull origin main
git checkout -b hooria          # your own branch (docs/TEAM_PLAN.md)
```

Then start the server and the website in two terminals:

```
# Terminal 1: server
cd server
npm install
copy .env.example .env          # Mac/Linux: cp .env.example .env
                                # open .env and set DB_USER / DB_PASSWORD to your MySQL login
npm run db:setup                # creates the database with sample data
node db/demo-data.js            # optional: adds past loans, repairs and notifications to test with
npm run dev                     # server on http://localhost:5000, restarts when you save

# Terminal 2: website
cd client
npm install
npm run dev                     # open http://localhost:5173
```

Log in as	Email	Password
Admin	`admin@cowork.local`	`Admin@123`
Member	`member@cowork.local`	`Member@123`

About the AI key

The AI features work in two modes, and you should test your screens in both:

In `server/.env`	Assistant chat	Recommendation
`ANTHROPIC_API_KEY=` (empty)	Answers with the error "The AI assistant is unavailable right now..." (status 503). Your screen must show it inside the chat.	Still works, using keyword ranking. The response has `source: "rules"`.
`ANTHROPIC_API_KEY=sk-ant-...`	Real answers with links to assets.	Real AI ranking, `source: "ai"`.

Ask Shahan for the team's key if you want to try the real AI. Never commit `.env` or paste the key into code.

How to test an endpoint without a screen (Postman or Thunder Client)

1. Send `POST http://localhost:5000/api/auth/login` with the JSON body `{ "email": "member@cowork.local", "password": "Member@123" }`.
2. Copy the `token` from the reply.
3. For every other request add the header `Authorization: Bearer <token>`.

Three things every server file in this project uses

Code	What it gives you
`const db = require('../../config/db')`	The shared MySQL connection. `const [rows] = await db.query(sql, [values])` runs a query. Always use `?` placeholders, never glue values into the SQL string (that is how SQL injection happens).
`router.use(requireAuth)`	Rejects anyone not logged in, and sets `req.user = { user_id, role, name }` for everyone else.
`throw new ApiError(404, 'Not found')`	Sends `{ "error": "Not found" }` with that status. Express 5 catches errors in `async` handlers for you, so no try/catch is needed.

1 Notifications API FR-NT-01, FR-NT-03

Open `server/src/modules/notifications/routes.js` and **replace everything** with:

server/src/modules/notifications/routes.js — 64 lines · complete file

```
// Notifications (FR-NT-01 to FR-NT-03)
// Owner: Hooria. Mounted at /api/notifications in app.js - do not rename this file.
// Every query uses req.user.user_id, so a user only ever sees and changes their own notifications.
const express = require('express');
const db = require('../../config/db');
const { requireAuth } = require('../../middleware/auth');
const { ApiError } = require('../../utils/http');

const router = express.Router();
router.use(requireAuth);

const LIST_LIMIT = 100; // newest 100 is plenty for the page

// FR-NT-01. In: -  Out: [{ notification_id, type, message, is_read, created_at }]  (newest first)
router.get('/', async (req, res) => {
  const [rows] = await db.query(
    `SELECT notification_id, type, message, is_read, created_at
       FROM notifications
      WHERE user_id = ?
      ORDER BY created_at DESC, notification_id DESC
      LIMIT ?`,
    [req.user.user_id, LIST_LIMIT],
  );
  res.json(rows.map((n) => ({ ...n, is_read: Boolean(n.is_read) })));
});

// FR-NT-01. In: -  Out: { count }
router.get('/unread-count', async (req, res) => {
  const [[row]] = await db.query(
    'SELECT COUNT(*) AS count FROM notifications WHERE user_id = ? AND is_read = FALSE',
    [req.user.user_id],
  );
  res.json({ count: Number(row.count) });
});

// FR-NT-03. In: -  Out: { message }
// Declared before '/:id/read' so "read-all" is never taken for an id.
router.patch('/read-all', async (req, res) => {
  const [result] = await db.query(
    'UPDATE notifications SET is_read = TRUE WHERE user_id = ? AND is_read = FALSE',
    [req.user.user_id],
  );
  const n = result.affectedRows;
  res.json({
    message: n
      ? `Marked ${n} notification${n === 1 ? '' : 's'} as read`
      : 'Everything was already read',
  });
});

// FR-NT-03. In: -  Out: { message }
router.patch('/:id/read', async (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id <= 0) throw new ApiError(400, 'Invalid notification id');
  const [result] = await db.query(
    'UPDATE notifications SET is_read = TRUE WHERE notification_id = ? AND user_id = ?',
    [id, req.user.user_id],
  );
  // Someone else's id looks exactly like a missing one, so nothing leaks.
  if (!result.affectedRows) throw new ApiError(404, 'Notification not found');
  res.json({ message: 'Marked as read' });
});

module.exports = router;
```

What the code does

- **Every query has `WHERE user_id = ?` with `req.user.user_id`.** That is the security rule: a user can only see and change their own notifications, even if they type another id into the URL.
- **The list** is newest first (`ORDER BY created_at DESC`) and limited to the latest 100. MySQL stores `is_read` as 0/1, so `Boolean(n.is_read)` turns it into `true` / `false` for the screen.
- **`/read-all` is written before `/:id/read`.** Express checks routes from top to bottom, so this order guarantees the word "read-all" is never treated as an id.
- **Mark one as read** checks the id is a positive number (400 if not), and returns 404 if nothing was updated. Someone else's notification looks exactly like a missing one, so nothing leaks.

Test it (Postman, logged in as the member)

Send	You should get
`GET /api/notifications`	A list (may be empty) of `{ notification_id, type, message, is_read, created_at }`
`GET /api/notifications/unread-count`	`{ "count": 2 }` (some number)
`PATCH /api/notifications/3/read`	`{ "message": "Marked as read" }`, or 404 if 3 is not yours
`PATCH /api/notifications/abc/read`	400 "Invalid notification id"
`PATCH /api/notifications/read-all`	"Marked 2 notifications as read", then the count is 0

No notifications yet? Log in as the member in the browser and request an asset: every admin gets a `new_request` notification. Approve it as the admin and the member gets `request_approved`.

2 "Due tomorrow" reminder job FR-NT-01

The SRS says a borrower is reminded one day before the due date. `server.js` already calls `start()` from this file when the server boots. **Open** `server/src/modules/notifications/jobs.js` and **replace everything** with:

server/src/modules/notifications/jobs.js — 45 lines · complete file

```
// Background job: remind borrowers one day before the due date (FR-NT-01).
// Owner: Hooria. server.js calls start() once when the server boots; it runs at once and then every hour.
const db = require('../../config/db');
const { notify } = require('../../services/notify');
const {
  OVERDUE_CHECK_INTERVAL_MS,
  DUE_REMINDER_LEAD_DAYS,
  BORROW_STATUS,
} = require('../../config/constants');

const reminderText = (assetName) =>
  `Reminder: ${assetName} is due back tomorrow. Please return it to the front desk.`;

// Returns how many reminders were sent. The job runs every hour, so the NOT EXISTS part skips
// a loan whose reminder was already sent today: each borrower is reminded once per loan.
async function sendDueReminders() {
  const [loans] = await db.query(
    `SELECT b.borrow_id, b.user_id, a.name AS asset_name
       FROM borrow_records b JOIN assets a ON a.asset_id = b.asset_id
      WHERE b.status = ? AND b.returned_at IS NULL
        AND b.due_date = CURDATE() + INTERVAL ? DAY`,
    [BORROW_STATUS.ACTIVE, DUE_REMINDER_LEAD_DAYS],
  );
  let sent = 0;
  for (const b of loans) {
    const message = reminderText(b.asset_name);
    const [[already]] = await db.query(
      `SELECT COUNT(*) AS n FROM notifications
        WHERE user_id = ? AND type = 'due_reminder' AND message = ? AND created_at >= CURDATE()`,
      [b.user_id, message],
```

```
    );
    if (already.n > 0) continue;
    await notify(b.user_id, 'due_reminder', message);
    sent += 1;
  }
  return sent;
}

function start() {
  const run = () => sendDueReminders().catch(console.error);
  run();
  setInterval(run, OVERDUE_CHECK_INTERVAL_MS);
}

module.exports = { start, sendDueReminders };
```

What the code does

- The first query finds loans that are still out (`status = 'Active'`, not returned) and due **tomorrow**: `CURDATE() + INTERVAL 1 DAY`, where 1 comes from `DUE_REMINDER_LEAD_DAYS` in `config/constants.js`.
- The job runs **every hour**, so without a check the borrower would get 24 reminders. The second query skips a loan when the same reminder was already sent today. Result: one reminder per loan.
- `notify(userId, type, message)` is the shared helper that inserts the notification row. The type `due_reminder` is what your Notifications page uses to pick the calendar icon.
- `start()` runs once straight away and then every hour (`OVERDUE_CHECK_INTERVAL_MS`). Errors are logged, never allowed to crash the server.

Test it

1. Find an active loan and make it due tomorrow (MySQL Workbench or phpMyAdmin):

   ```
   SELECT borrow_id, user_id, due_date FROM borrow_records WHERE status = 'Active';
   UPDATE borrow_records SET due_date = CURDATE() + INTERVAL 1 DAY WHERE borrow_id = 30;  -- use an
   id from above
   ```

2. Restart the server (save any file, or Ctrl+C and `npm run dev`). The job runs at start-up.
3. Log in as that loan's borrower: there is one "Reminder: ... is due back tomorrow" notification. Restart again: still only one.

3 The bell in the top bar FR-NT-01

`components/Layout.jsx` already shows this component on every page, so you only fill it in. **Open** `client/src/modules/notifications/NotificationBell.jsx` and **replace everything** with:

client/src/modules/notifications/NotificationBell.jsx — 55 lines · complete file

```jsx
// Owner: Hooria. Shown in the top bar on every page (components/Layout.jsx imports it).
// Polls GET /api/notifications/unread-count every 60 s (NOTIFICATION_POLL, FR-NT-01) and shows the
// count on the bell. The Notifications page sends a 'notifications:changed' event after marking
// something read, so the number drops at once instead of waiting for the next poll.
import { useCallback, useEffect, useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { Bell } from 'lucide-react';
import api from '../../api/client';
import { Button } from '../../components/ui';

const POLL_MS = 60 * 1000;
export const NOTIFICATIONS_CHANGED = 'notifications:changed';

export default function NotificationBell() {
  const [count, setCount] = useState(0);
  const { pathname } = useLocation();

  const refresh = useCallback(async () => {
    try {
      const data = await api.get('/notifications/unread-count');
      setCount(data.count);
    } catch {
      // A failed poll is not worth an error on every page; the next poll tries again.
    }
  }, []);

  useEffect(() => {
    refresh();
    const timer = setInterval(refresh, POLL_MS);
    window.addEventListener(NOTIFICATIONS_CHANGED, refresh);
    return () => {
      clearInterval(timer);
      window.removeEventListener(NOTIFICATIONS_CHANGED, refresh);
    };
  }, [refresh]);

  // Also check when the user moves to another page.
  useEffect(() => {
    refresh();
  }, [pathname, refresh]);

  const label = count ? `Notifications, ${count} unread` : 'Notifications';
  return (
    <Button asChild variant="secondary" size="icon" className="border-border relative border">
      <Link to="/notifications" title={label} aria-label={label}>
        <Bell />
        {count > 0 && (
          <span className="bg-primary text-primary-foreground absolute -top-1 -right-1 flex h-5 min-w-5
items-center justify-center rounded-full px-1 text-[0.65rem] font-semibold tabular-nums shadow-[0_0_12px]
shadow-primary">
            {count > 99 ? '99+' : count}
          </span>
        )}
      </Link>
    </Button>
  );
}
```

What the code does

- **`useEffect` with `setInterval`** asks the server for the unread count every 60 seconds (the SRS constant `NOTIFICATION_POLL`). The `return () => clearInterval(...)` stops the timer when the component goes away, so timers don't pile up.

- **It also refreshes when the page changes** (`useLocation().pathname`), so moving around the app shows new notifications quickly.
- `NOTIFICATIONS_CHANGED` is a small browser event. The Notifications page fires it after marking something read, and the bell listens for it, so the number drops at once instead of up to a minute later.
- The violet badge only shows when the count is above 0, and "99+" stops it from getting too wide. A failed poll is ignored on purpose: an error box on every page would be annoying, and the next poll tries again.

Test it

Log in as the admin after the member has requested something: the bell shows a number. Click it: it opens `/notifications` (still a placeholder until step 4).

4 Notifications page FR-NT-01 to 03

Open `client/src/modules/notifications/NotificationsPage.jsx` and **replace everything** with:

client/src/modules/notifications/NotificationsPage.jsx — 229 lines · complete file

```
// Owner: Hooria. Route: /notifications (registered in src/routes.jsx - do not rename this file or its
export).
// Notifications (FR-NT-01 to FR-NT-03): the user's in-app messages, newest first. Unread ones are
// highlighted; the user can mark one or all as read. Opening a message marks it read and goes to
// the screen where they can act on it.
import { useCallback, useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { toast } from 'sonner';
import {
  AlarmClock,
  Bell,
  CalendarClock,
  CheckCheck,
  CheckCircle2,
  ClipboardList,
  PackageCheck,
  PackageOpen,
  RefreshCw,
  UserCheck,
  Wrench,
  XCircle,
} from 'lucide-react';
import api from '../../api/client';
import { useAuth } from '../../context/AuthContext';
import {
  Button,
  Card,
  CardContent,
  EmptyState,
  ErrorBox,
  Page,
  Skeleton,
  Tabs,
  TabsList,
  TabsTrigger,
} from '../../components/ui';
import { cn } from '@/lib/utils';
import { fmtDate } from '../reports/format';
import { NOTIFICATIONS_CHANGED } from './NotificationBell';

// One icon, colour and target screen for every notification type the other modules send (docs/API.md).
const TYPES = {
  new_request: { icon: ClipboardList, tone: 'text-warning', to: '/admin/approvals' },
  problem_report: { icon: Wrench, tone: 'text-danger', to: '/admin/maintenance' },
  request_approved: { icon: CheckCircle2, tone: 'text-success', to: '/my-requests' },
  request_rejected: { icon: XCircle, tone: 'text-danger', to: '/my-requests' },
  request_cancelled: { icon: XCircle, tone: 'text-muted-foreground', to: '/my-requests' },
  asset_issued: { icon: PackageOpen, tone: 'text-primary', to: '/my-requests' },
  asset_returned: { icon: PackageCheck, tone: 'text-success', to: '/history' },
  due_reminder: { icon: CalendarClock, tone: 'text-warning', to: '/my-requests' },
  overdue: { icon: AlarmClock, tone: 'text-danger', to: '/my-requests' },
  problem_resolved: { icon: Wrench, tone: 'text-success', to: '/history' },
  account_reactivated: { icon: UserCheck, tone: 'text-success', to: null },
};
const DEFAULT_TYPE = { icon: Bell, tone: 'text-muted-foreground', to: null };

// "2026-10-24 10:08:50" -> "24 Oct 2026, 10:08"
const fmtWhen = (value) => `${fmtDate(value)}, ${String(value).slice(11, 16)}`;

export default function NotificationsPage() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const [rows, setRows] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [filter, setFilter] = useState('All');
```

```
const load = useCallback(async () => {
  setLoading(true);
  setError(null);
  try {
    setRows(await api.get('/notifications'));
  } catch (err) {
    setError(err);
  } finally {
    setLoading(false);
  }
}, []);

useEffect(() => {
  load();
}, [load]);

const unread = rows.filter((n) => !n.is_read).length;
const shown = filter === 'Unread' ? rows.filter((n) => !n.is_read) : rows;

// Updates the list on screen straight away and tells the bell to recount.
function markLocally(id) {
  setRows((list) =>
    list.map((n) => (id === 'all' || n.notification_id === id ? { ...n, is_read: true } : n)),
  );
  window.dispatchEvent(new Event(NOTIFICATIONS_CHANGED));
}

async function markRead(n) {
  try {
    await api.patch(`/notifications/${n.notification_id}/read`);
    markLocally(n.notification_id);
  } catch (err) {
    toast.error(err.message);
  }
}

async function markAll() {
  try {
    const { message } = await api.patch('/notifications/read-all');
    markLocally('all');
    toast.success(message);
  } catch (err) {
    toast.error(err.message);
  }
}

async function open(n) {
  if (!n.is_read) await markRead(n);
  const target = (TYPES[n.type] || DEFAULT_TYPE).to;
  // Members have no admin screens and admins have no member screens.
  if (target && target.startsWith('/admin') === (user.role === 'Admin')) navigate(target);
}

return (
  <Page
    title="Notifications"
    accent={unread ? `${unread} new` : undefined}
    description="Updates about your requests, due dates and problem reports."
    actions={
      <>
        <Button
          variant="outline"
          size="icon"
          onClick={load}
          disabled={loading}
          aria-label="Refresh"
        >
          <RefreshCw className={loading ? 'animate-spin' : undefined} />
        </Button>
        <Button variant="brand" onClick={markAll} disabled={unread === 0}>
          <CheckCheck /> Mark all as read
        </Button>
```

```
        </>
      }
    >
      <ErrorBox error={error} />

      <Tabs value={filter} onValueChange={setFilter}>
        <TabsList>
          <TabsTrigger value="All">
            All <span className="text-muted-foreground text-xs tabular-nums">{rows.length}</span>
          </TabsTrigger>
          <TabsTrigger value="Unread">
            Unread <span className="text-muted-foreground text-xs tabular-nums">{unread}</span>
          </TabsTrigger>
        </TabsList>
      </Tabs>

      <Card className="overflow-hidden py-0">
        <CardContent className="p-0">
          {loading && rows.length === 0 ? (
            <div className="space-y-4 p-6">
              {[1, 2, 3].map((i) => (
                <Skeleton key={i} className="h-12 w-full rounded-2xl" />
              ))}
            </div>
          ) : shown.length === 0 ? (
            <EmptyState
              icon={Bell}
              title={filter === 'Unread' ? 'You are all caught up' : 'No notifications yet'}
            >
              {filter === 'Unread'
                ? 'Nothing new since you last looked.'
                : 'You will see updates about your requests and loans here.'}
            </EmptyState>
          ) : (
            <ul className="divide-border divide-y">
              {shown.map((n) => {
                const t = TYPES[n.type] || DEFAULT_TYPE;
                const Icon = t.icon;
                return (
                  <li
                    key={n.notification_id}
                    className={cn(
                      'flex items-start gap-4 px-5 py-4 transition-colors',
                      !n.is_read && 'bg-primary/8',
                    )}
                  >
                    <span
                      className={cn(
                        'border-border flex size-10 shrink-0 items-center justify-center rounded-full
border',
                        t.tone,
                      )}
                    >
                      <Icon className="size-4" />
                    </span>
                    <button
                      type="button"
                      onClick={() => open(n)}
                      className="min-w-0 flex-1 text-left"
                    >
                      <p
                        className={cn(
                          'text-sm',
                          n.is_read ? 'text-muted-foreground' : 'text-foreground font-medium',
                        )}
                      >
                        {n.message}
                      </p>
                      <p className="text-muted-foreground mt-1 text-xs">{fmtWhen(n.created_at)}</p>
                    </button>
                    {!n.is_read && (
                      <div className="flex shrink-0 items-center gap-2">
                        <span
```

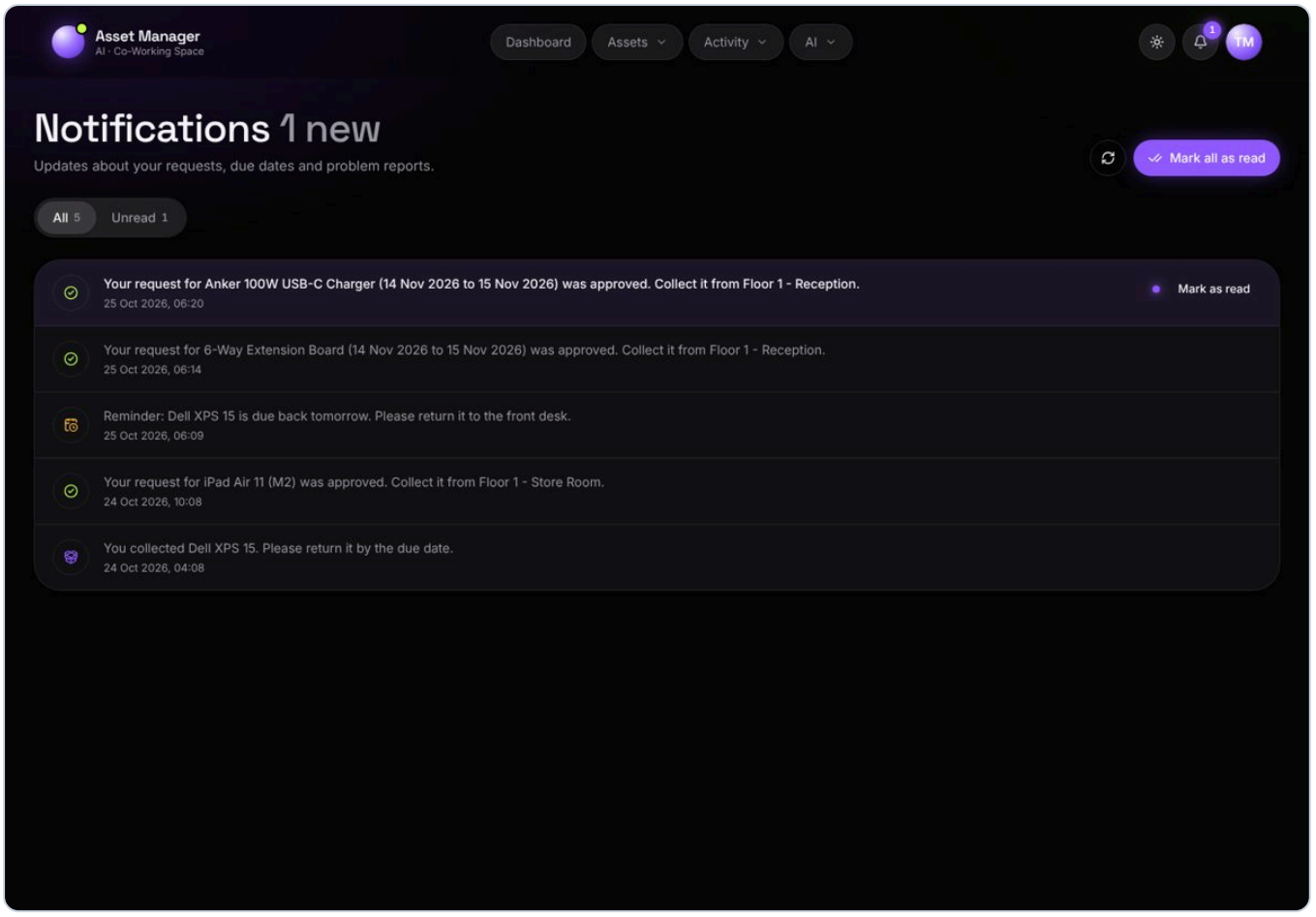
```
                        className="bg-primary size-2 rounded-full shadow-[0_0_10px] shadow-primary"
                        aria-label="Unread"
                      />
                      <Button size="sm" variant="ghost" onClick={() => markRead(n)}>
                        <span className="hidden sm:inline">Mark as read</span>
                        <CheckCheck className="sm:hidden" />
                      </Button>
                    </div>
                  )}
                </li>
              );
            })}
          </ul>
        )}
      </CardContent>
    </Card>
  </Page>
 );
}
```

What the code does

- `TYPES` gives every notification type an icon, a colour and the screen where the user can act on it. These are the types Hamza's and Azlaan's code send (listed in `docs/API.md`). An unknown type falls back to a plain bell, so a new type never breaks the page.
- `load()` is the standard pattern from `PageTemplate.jsx`: set loading, call the API, keep the rows or the error, stop loading.
- `markLocally()` updates the list on screen straight away (no reload) and fires `NOTIFICATIONS_CHANGED` so the bell recounts.
- **Clicking a message** marks it read and opens the right screen, for example a "request approved" message opens My Requests. Admin links are only followed by admins, member links only by members.
- **Tabs** switch between All and Unread. Unread rows have a light violet background, bold text, a glowing dot and a "Mark as read" button. "Mark all as read" is disabled when nothing is unread.

Test it

1. As the member, open the bell. Unread messages are highlighted; the title says "Notifications 2 new".
2. Click "Mark as read" on one: the highlight goes and the bell number drops by one at once.
3. Click "Mark all as read": a green toast appears and the bell badge disappears.
4. As the admin, approve a request of the member's. Within a minute (or on the next page change) the member's bell shows 1 again.



The Notifications page (member, dark mode). The newest message is unread.

5 History API FR-UH-01 to 03

Open `server/src/modules/history/routes.js` and **replace everything** with:

server/src/modules/history/routes.js — 81 lines · complete file

```
// Usage History (FR-UH-01 to FR-UH-03)
// Owner: Hooria. Mounted at /api/history in app.js - do not rename this file.
// Read only (FR-UH-03): this file has GET routes and nothing else, so history can never be edited.
const express = require('express');
const db = require('../../config/db');
const { requireAuth, requireRole } = require('../../middleware/auth');
const { ApiError } = require('../../utils/http');

const router = express.Router();
router.use(requireAuth);
const admin = requireRole('Admin');

// FR-UH-02. The logged-in user's own requests and borrowings, newest first.
// Out: { requests: [{ request_id, asset_id, asset_tag, asset_name, category, purpose, start_date, end_date,
//                     status, rejection_reason, created_at, decided_at }],
//        borrowings: [{ borrow_id, request_id, asset_id, asset_tag, asset_name, category, issued_at,
due_date,
//                       returned_at, return_condition, status }] }
router.get('/mine', async (req, res) => {
  const [requests] = await db.query(
    `SELECT r.request_id, a.asset_id, a.asset_tag, a.name AS asset_name, c.name AS category,
            r.purpose, r.start_date, r.end_date, r.status, r.rejection_reason, r.created_at, r.decided_at
       FROM asset_requests r
       JOIN assets a ON a.asset_id = r.asset_id
       JOIN categories c ON c.category_id = a.category_id
      WHERE r.user_id = ?
      ORDER BY r.created_at DESC, r.request_id DESC`,
    [req.user.user_id],
  );
  const [borrowings] = await db.query(
    `SELECT b.borrow_id, b.request_id, a.asset_id, a.asset_tag, a.name AS asset_name, c.name AS category,
            b.issued_at, b.due_date, b.returned_at, b.return_condition, b.status
       FROM borrow_records b
       JOIN assets a ON a.asset_id = b.asset_id
       JOIN categories c ON c.category_id = a.category_id
      WHERE b.user_id = ?
      ORDER BY b.issued_at DESC, b.borrow_id DESC`,
    [req.user.user_id],
  );
  res.json({ requests, borrowings });
});

// FR-UH-01 (admin only). Everything that happened to one asset.
// Out: { asset: { asset_id, asset_tag, name, category, location, condition, status, image_path },
//        borrowings: [{ borrow_id, user_id, user_name, user_email, issued_at, due_date, returned_at,
return_condition, status }],
//        maintenance: [{ maintenance_id, issue_description, notes, cost, status, outcome, reported_by_name,
reported_at, resolved_at }] }
router.get('/asset/:id', admin, async (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id <= 0) throw new ApiError(400, 'Invalid asset id');

  const [[asset]] = await db.query(
    `SELECT a.asset_id, a.asset_tag, a.name, c.name AS category, a.location, a.\`condition\`, a.status,
a.image_path
       FROM assets a JOIN categories c ON c.category_id = a.category_id
      WHERE a.asset_id = ?`,
    [id],
  );
  if (!asset) throw new ApiError(404, 'This asset does not exist');

  const [borrowings] = await db.query(
    `SELECT b.borrow_id, u.user_id, u.name AS user_name, u.email AS user_email,
            b.issued_at, b.due_date, b.returned_at, b.return_condition, b.status
       FROM borrow_records b JOIN users u ON u.user_id = b.user_id
      WHERE b.asset_id = ?
```

```
      ORDER BY b.issued_at DESC, b.borrow_id DESC`,
    [id],
  );
  const [maintenance] = await db.query(
    `SELECT m.maintenance_id, m.issue_description, m.notes, m.cost, m.status, m.outcome,
            u.name AS reported_by_name, m.reported_at, m.resolved_at
       FROM maintenance_logs m JOIN users u ON u.user_id = m.reported_by
      WHERE m.asset_id = ?
      ORDER BY m.reported_at DESC, m.maintenance_id DESC`,
    [id],
  );
  res.json({
    asset,
    borrowings,
    maintenance: maintenance.map((m) => ({ ...m, cost: Number(m.cost) })),
  });
});

module.exports = router;
```

What the code does

- **Read only (FR-UH-03):** the file has `GET` routes and nothing else, so there is no way to edit or delete history through the API. Mention this in the viva: the rule is enforced on the server, not only by hiding buttons.
- `/mine` runs two queries for the logged-in user: their requests from `asset_requests` and their loans from `borrow_records`. Both `JOIN` the asset and its category so the screen can show names, not just ids.
- `/asset/:id` has the `admin` middleware, so a member gets 403. It checks the asset exists (404 if not), then loads every loan with the borrower's name and email, and every maintenance record with who reported it.
- MySQL returns `DECIMAL` money as a string such as `"1500.00"`; `Number(m.cost)` makes it a real number so the screen can add costs up.

Test it

Send	You should get
`GET /api/history/mine` (member)	`{ requests: [...], borrowings: [...] }`
`GET /api/history/asset/1` (admin)	`{ asset: {...}, borrowings: [...], maintenance: [...] }`
`GET /api/history/asset/1` (member)	403 "You do not have permission to do this"
`GET /api/history/asset/9999` (admin)	404 "This asset does not exist"

6 My History page FR-UH-02, FR-UH-03

Open `client/src/modules/history/MyHistoryPage.jsx` and **replace everything** with:

client/src/modules/history/MyHistoryPage.jsx — 236 lines · complete file

```jsx
// Owner: Hooria. Route: /history (registered in src/routes.jsx - do not rename this file or its export).
// My History (FR-UH-02, FR-UH-03): every request the member made and every item they borrowed.
// Read only: there are no edit or delete buttons on this screen.
import { useCallback, useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { ClipboardList, History, PackageCheck, PackageOpen, RefreshCw } from 'lucide-react';
import api from '../../api/client';
import {
  Badge,
  Button,
  Card,
  EmptyState,
  ErrorBox,
  Page,
  StatCard,
  StatusBadge,
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
  TableSkeleton,
  Tabs,
  TabsContent,
  TabsList,
  TabsTrigger,
} from '../../components/ui';
import { fmtDate } from '../reports/format';

// Main value with a small grey line under it.
function AssetCell({ row }) {
  return (
    <>
      <Link to={`/assets/${row.asset_id}`} className="font-medium hover:underline">
        {row.asset_name}
      </Link>
      <p className="text-muted-foreground text-xs">
        <span className="font-mono">{row.asset_tag}</span> · {row.category}
      </p>
    </>
  );
}

function RequestsTable({ rows }) {
  if (rows.length === 0) {
    return (
      <EmptyState icon={ClipboardList} title="No requests yet">
        Requests you send appear here, with their result.
      </EmptyState>
    );
  }
  return (
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>Asset</TableHead>
          <TableHead className="hidden sm:table-cell">Dates</TableHead>
          <TableHead className="hidden lg:table-cell">Purpose</TableHead>
          <TableHead>Status</TableHead>
          <TableHead className="hidden md:table-cell">Sent</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {rows.map((r) => (
          <TableRow key={r.request_id}>
```

```
            <TableCell>
              <AssetCell row={r} />
            </TableCell>
            <TableCell className="hidden whitespace-nowrap sm:table-cell">
              {fmtDate(r.start_date)} to {fmtDate(r.end_date)}
            </TableCell>
            <TableCell
              className="text-muted-foreground hidden max-w-64 truncate lg:table-cell"
              title={r.purpose}
            >
              {r.purpose}
            </TableCell>
            <TableCell>
              <StatusBadge status={r.status} />
              {r.rejection_reason && (
                <p className="text-muted-foreground mt-1 max-w-56 text-xs whitespace-normal">
                  {r.rejection_reason}
                </p>
              )}
            </TableCell>
            <TableCell className="text-muted-foreground hidden whitespace-nowrap md:table-cell">
              {fmtDate(r.created_at)}
            </TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  );
}

function BorrowingsTable({ rows }) {
  if (rows.length === 0) {
    return (
      <EmptyState icon={PackageOpen} title="Nothing borrowed yet">
        Items you collect from the front desk appear here.
      </EmptyState>
    );
  }
  return (
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>Asset</TableHead>
          <TableHead className="hidden sm:table-cell">Collected</TableHead>
          <TableHead className="hidden sm:table-cell">Due</TableHead>
          <TableHead>Returned</TableHead>
          <TableHead className="hidden md:table-cell">Condition</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {rows.map((b) => (
          <TableRow key={b.borrow_id}>
            <TableCell>
              <AssetCell row={b} />
            </TableCell>
            <TableCell className="hidden whitespace-nowrap sm:table-cell">
              {fmtDate(b.issued_at)}
            </TableCell>
            <TableCell className="hidden whitespace-nowrap sm:table-cell">
              {fmtDate(b.due_date)}
            </TableCell>
            <TableCell className="whitespace-nowrap">
              {b.returned_at ? (
                fmtDate(b.returned_at)
              ) : (
                <StatusBadge status={b.status === 'Overdue' ? 'Overdue' : 'Borrowed'} />
              )}
            </TableCell>
            <TableCell className="hidden md:table-cell">
              {b.return_condition ? (
                <Badge variant={b.return_condition === 'Damaged' ? 'danger' : 'pill'}>
                  {b.return_condition}
                </Badge>
```

```
              ) : (
                '-'
              )}
            </TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  );
}

export default function MyHistoryPage() {
  const [data, setData] = useState({ requests: [], borrowings: [] });
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const load = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      setData(await api.get('/history/mine'));
    } catch (err) {
      setError(err);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    load();
  }, [load]);

  const { requests, borrowings } = data;
  const returned = borrowings.filter((b) => b.returned_at).length;
  const onLoan = borrowings.length - returned;

  return (
    <Page
      title="My"
      accent="history"
      description="Everything you have requested and borrowed. This record cannot be changed."
      actions={
        <Button
          variant="outline"
          size="icon"
          onClick={load}
          disabled={loading}
          aria-label="Refresh"
        >
          <RefreshCw className={loading ? 'animate-spin' : undefined} />
        </Button>
      }
    >
      <ErrorBox error={error} />

      <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
        <StatCard label="Requests sent" value={requests.length} icon={ClipboardList} tone="brand" />
        <StatCard label="Times borrowed" value={borrowings.length} icon={History} tone="info" />
        <StatCard label="Returned" value={returned} icon={PackageCheck} tone="success" />
        <StatCard
          label="On loan now"
          value={onLoan}
          icon={PackageOpen}
          tone={onLoan ? 'warning' : 'neutral'}
        />
      </div>

      <Tabs defaultValue="requests">
        <TabsList>
          <TabsTrigger value="requests">
            Requests{' '}
            <span className="text-muted-foreground text-xs tabular-nums">{requests.length}</span>
          </TabsTrigger>
```

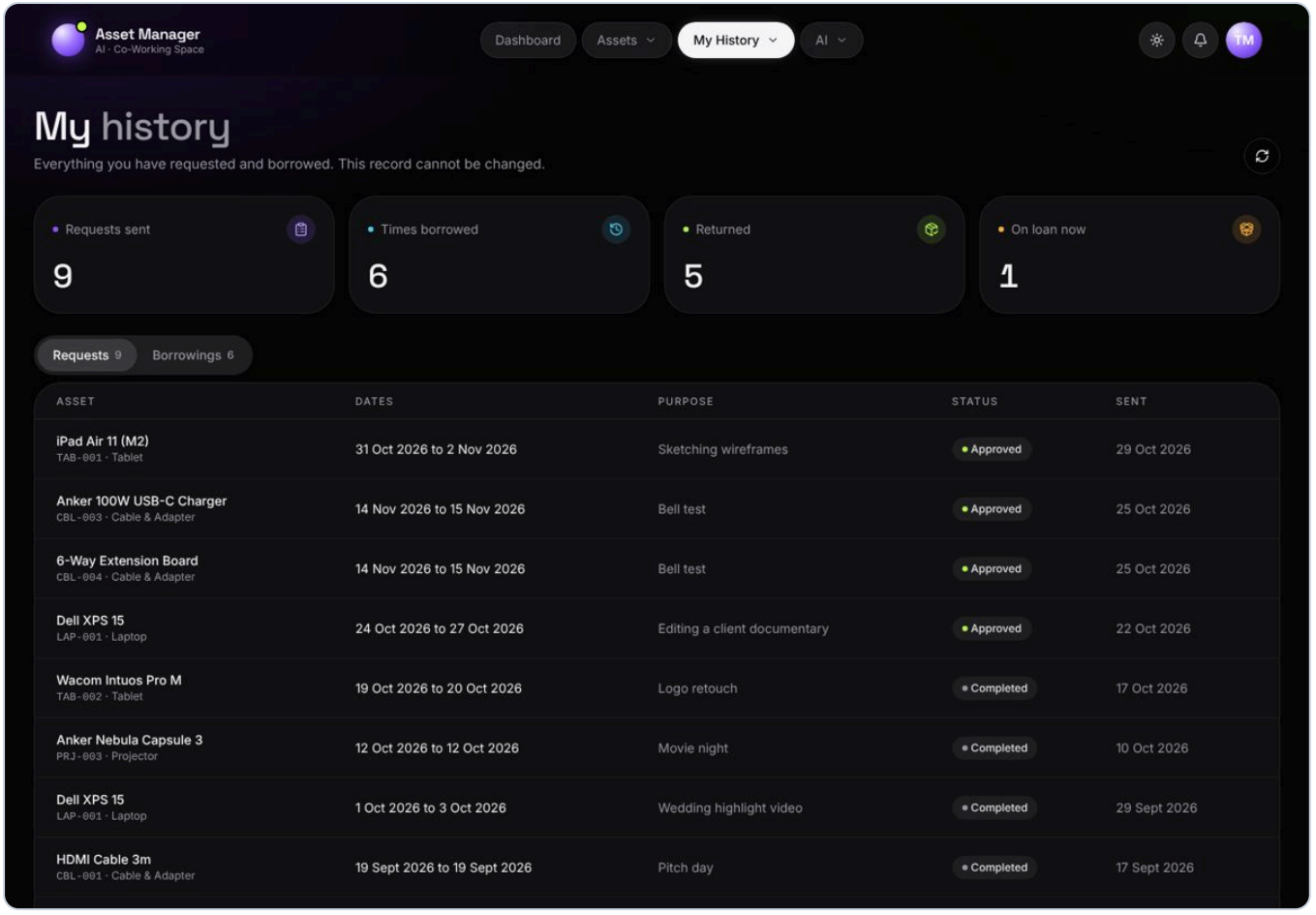
```
          <TabsTrigger value="borrowings">
            Borrowings{' '}
            <span className="text-muted-foreground text-xs tabular-nums">{borrowings.length}</span>
          </TabsTrigger>
        </TabsList>
        {['requests', 'borrowings'].map((tab) => (
          <TabsContent key={tab} value={tab}>
            <Card className="overflow-hidden py-0">
              {loading && requests.length + borrowings.length === 0 ? (
                <div className="py-4">
                  <TableSkeleton />
                </div>
              ) : tab === 'requests' ? (
                <RequestsTable rows={requests} />
              ) : (
                <BorrowingsTable rows={borrowings} />
              )}
            </Card>
          </TabsContent>
        ))}
      </Tabs>
    </Page>
  );
}
```

What the code does

- One API call (`/history/mine`) fills the whole page: four `StatCard`s at the top, then **Tabs** with a Requests table and a Borrowings table.
- `RequestsTable` and `BorrowingsTable` are small components in the same file. Each shows an `EmptyState` when it has no rows. Asset names link to Azlaan's asset details page.
- Columns marked `hidden sm:table-cell` disappear on a phone so the table fits; `StatusBadge` picks the right colour for every status word.
- `fmtDate` comes from Misbah's `reports/format.js`, so dates look the same everywhere ("24 Oct 2026").
- There are no edit or delete buttons anywhere (FR-UH-03).

Test it

Log in as the member and open **Activity → My History**. Switch between the two tabs. Try the phone width in Chrome (F12, then the phone icon, 390 px wide): the page must not scroll sideways.



My History (member). The Borrowings tab shows collected, due and returned dates and the return condition.

7 Asset History page (admin) FR-UH-01, FR-UH-03

Azlaan's Asset Details and Manage Assets screens already have a "View history" link to `/admin/history/<id>`. **Open** `client/src/modules/history/AssetHistoryPage.jsx` and **replace everything** with:

client/src/modules/history/AssetHistoryPage.jsx — 225 lines · complete file

```
// Owner: Hooria. Route: /admin/history/:assetId (registered in src/routes.jsx - do not rename this file or
its export).
// Asset History (FR-UH-01, FR-UH-03), admin only: every borrowing of one asset with the borrower's
// name, and every maintenance record. Opened from Azlaan's Asset Details and Manage Assets screens.
import { useCallback, useEffect, useState } from 'react';
import { Link, useParams } from 'react-router-dom';
import { ArrowLeft, History, PackageOpen, Wallet, Wrench } from 'lucide-react';
import api from '../../api/client';
import {
  Badge,
  Button,
  Card,
  CardContent,
  CardHeader,
  CardIcon,
  CardTitle,
  EmptyState,
  ErrorBox,
  Page,
  StatCard,
  StatusBadge,
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
  TableSkeleton,
} from '../../components/ui';
import { fmtDate, fmtMoney } from '../reports/format';
import { AssetThumb } from '../assets/parts';

export default function AssetHistoryPage() {
  const { assetId } = useParams();
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const load = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      setData(await api.get(`/history/asset/${assetId}`));
    } catch (err) {
      setError(err);
    } finally {
      setLoading(false);
    }
  }, [assetId]);

  useEffect(() => {
    load();
  }, [load]);

  const back = (
    <Button asChild variant="outline">
      <Link to={`/assets/${assetId}`}>
        <ArrowLeft /> Back to asset
      </Link>
    </Button>
  );

  if (loading && !data) {
```

```
    return (
      <Page title="Asset" accent="history" actions={back}>
        <Card>
          <TableSkeleton />
        </Card>
      </Page>
    );
  }
  if (!data) {
    return (
      <Page title="Asset" accent="history" actions={back}>
        <ErrorBox error={error} />
      </Page>
    );
  }

  const { asset, borrowings, maintenance } = data;
  const totalCost = maintenance.reduce((sum, m) => sum + m.cost, 0);
  const totalDays = borrowings.reduce((sum, b) => {
    const end = b.returned_at ? new Date(b.returned_at.slice(0, 10)) : new Date();
    return sum + Math.max(1, Math.round((end - new Date(b.issued_at.slice(0, 10))) / 86400000));
  }, 0);

  return (
    <Page
      title={asset.name}
      accent="history"
      description={`${asset.asset_tag} · ${asset.category} · ${asset.location}. History cannot be edited.`}
      actions={back}
    >
      <ErrorBox error={error} />

      <div className="grid gap-4 lg:grid-cols-[18rem_1fr]">
        <Card className="gap-3 p-4">
          <AssetThumb asset={asset} />
          <div className="flex flex-wrap gap-2">
            <StatusBadge status={asset.status} />
            <Badge variant="pill">Condition: {asset.condition}</Badge>
          </div>
        </Card>
        <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
          <StatCard label="Times borrowed" value={borrowings.length} icon={History} tone="brand" />
          <StatCard label="Days on loan" value={totalDays} icon={PackageOpen} tone="info" />
          <StatCard
            label="Repairs"
            value={maintenance.length}
            icon={Wrench}
            tone={maintenance.length ? 'warning' : 'neutral'}
          />
          <StatCard label="Repair cost" value={fmtMoney(totalCost)} icon={Wallet} tone="neutral" />
        </div>
      </div>

      <Card className="overflow-hidden pb-0">
        <CardHeader>
          <CardTitle>
            <CardIcon icon={History} /> Borrowings
          </CardTitle>
        </CardHeader>
        <CardContent className="p-0">
          {borrowings.length === 0 ? (
            <EmptyState icon={History} title="Never borrowed">
              Nobody has borrowed this asset yet.
            </EmptyState>
          ) : (
            <Table>
              <TableHeader>
                <TableRow>
                  <TableHead>Borrower</TableHead>
                  <TableHead className="hidden sm:table-cell">Collected</TableHead>
                  <TableHead className="hidden sm:table-cell">Due</TableHead>
                  <TableHead>Returned</TableHead>
                  <TableHead className="hidden md:table-cell">Condition</TableHead>
```

```
              </TableRow>
            </TableHeader>
            <TableBody>
              {borrowings.map((b) => (
                <TableRow key={b.borrow_id}>
                  <TableCell>
                    <p className="font-medium">{b.user_name}</p>
                    <p className="text-muted-foreground text-xs">{b.user_email}</p>
                  </TableCell>
                  <TableCell className="hidden whitespace-nowrap sm:table-cell">
                    {fmtDate(b.issued_at)}
                  </TableCell>
                  <TableCell className="hidden whitespace-nowrap sm:table-cell">
                    {fmtDate(b.due_date)}
                  </TableCell>
                  <TableCell className="whitespace-nowrap">
                    {b.returned_at ? (
                      fmtDate(b.returned_at)
                    ) : (
                      <StatusBadge status={b.status === 'Overdue' ? 'Overdue' : 'Borrowed'} />
                    )}
                  </TableCell>
                  <TableCell className="hidden md:table-cell">
                    {b.return_condition ? (
                      <Badge variant={b.return_condition === 'Damaged' ? 'danger' : 'pill'}>
                        {b.return_condition}
                      </Badge>
                    ) : (
                      '-'
                    )}
                  </TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
        )}
      </CardContent>
    </Card>

    <Card className="overflow-hidden pb-0">
      <CardHeader>
        <CardTitle>
          <CardIcon icon={Wrench} /> Maintenance
        </CardTitle>
      </CardHeader>
      <CardContent className="p-0">
        {maintenance.length === 0 ? (
          <EmptyState icon={Wrench} title="No repairs">
            This asset has never been sent for maintenance.
          </EmptyState>
        ) : (
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Problem</TableHead>
                <TableHead className="hidden sm:table-cell">Reported</TableHead>
                <TableHead>Status</TableHead>
                <TableHead className="text-right">Cost</TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {maintenance.map((m) => (
                <TableRow key={m.maintenance_id}>
                  <TableCell className="max-w-80 whitespace-normal">
                    <p className="font-medium">{m.issue_description}</p>
                    {m.notes && <p className="text-muted-foreground text-xs">{m.notes}</p>}
                  </TableCell>
                  <TableCell className="hidden whitespace-nowrap sm:table-cell">
                    {fmtDate(m.reported_at)}
                    <p className="text-muted-foreground text-xs">by {m.reported_by_name}</p>
                  </TableCell>
                  <TableCell>
                    <StatusBadge status={m.outcome || m.status} />
```

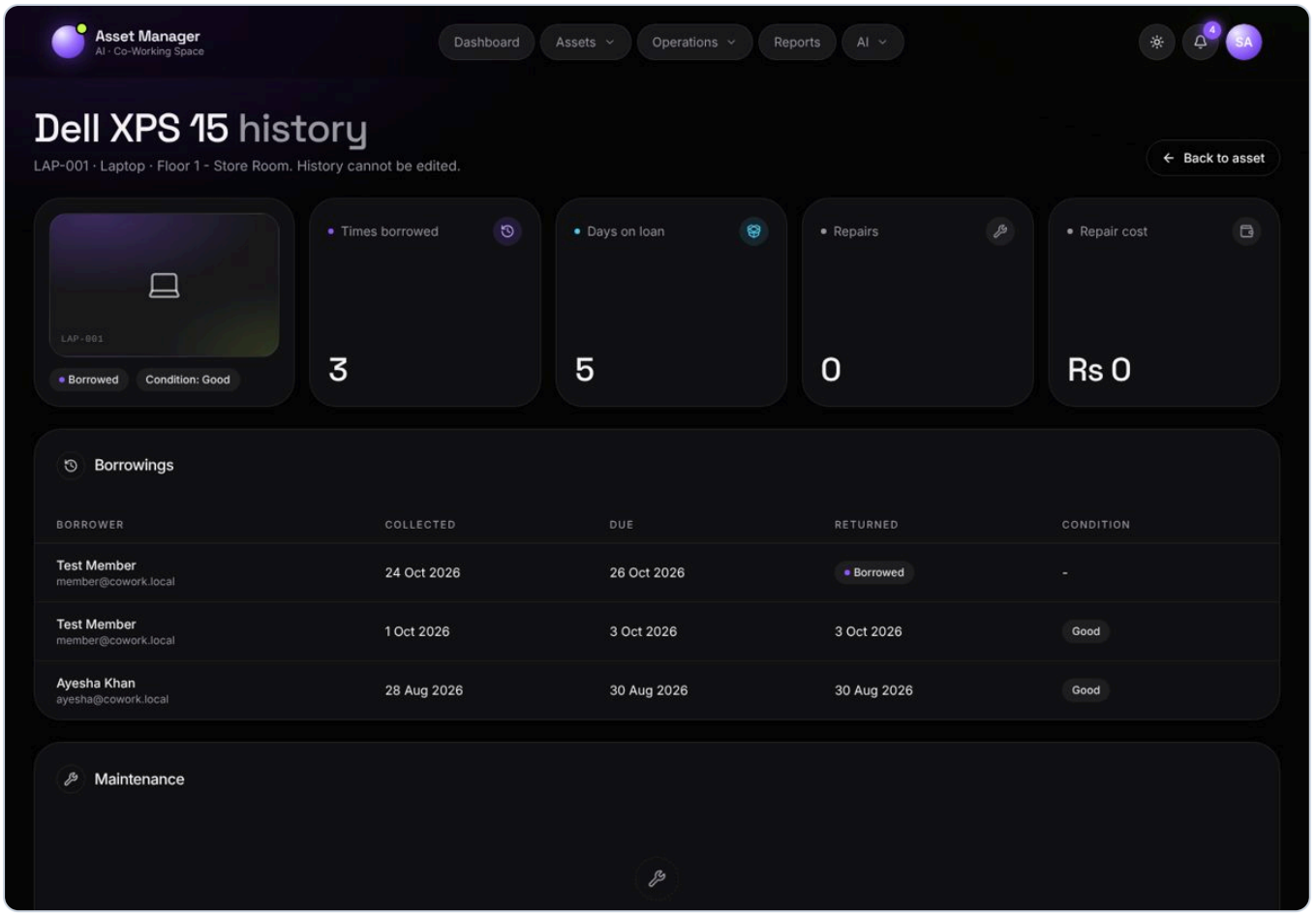
```
                      {m.resolved_at && (
                        <p className="text-muted-foreground mt-1 text-xs">
                          {fmtDate(m.resolved_at)}
                        </p>
                      )}
                    </TableCell>
                    <TableCell className="text-right tabular-nums">{fmtMoney(m.cost)}</TableCell>
                  </TableRow>
                ))}
              </TableBody>
            </Table>
          )}
        </CardContent>
      </Card>
    </Page>
  );
}
```

What the code does

- `useParams()` reads the asset id from the URL. Because `assetId` is listed in `useCallback(..., [assetId])`, the page reloads if the id changes.
- **Two early returns** handle loading (a skeleton) and a failed load (the error, e.g. "This asset does not exist"), so the rest of the code can safely use `data`.
- The top row shows the asset picture (`AssetThumb` from Azlaan's `assets/parts.jsx`, which draws a category icon when there is no photo) and four totals: times borrowed, days on loan, repairs and repair cost. They are worked out in the browser with `reduce`.
- Two cards follow: **Borrowings** (who, when, returned, condition) and **Maintenance** (problem, who reported it, status or outcome, cost). Both are read only.
- Members never reach this page: `routes.jsx` only allows admins and sends anyone else to their home page. The server's 403 is the second lock.

Test it

1. As the admin open **Assets → Asset Catalogue**, click an asset, then **View history**.
2. Type `/admin/history/9999` in the address bar: you see "This asset does not exist".
3. Log in as the member and type `/admin/history/1`: you are sent back to the member dashboard.



Asset History for Dell XPS 15 (admin).

8 AI Recommendation screen FR-RC-01, 04, 05, 06

Misbah's `POST /api/ai/recommend` does the thinking. Your screen sends the form and shows the answer:

```
// You send
{ requirement: "laptop for video editing", purpose: "Client video", start_date: "2026-11-14",
duration_days: 3 }
// You get back
{ source: "ai" or "rules", start_date, end_date, duration_days,
  recommendations: [ { rank, reason, asset: { asset_id, asset_tag, name, category, location,
condition, image_path } } ],
  message?, nearest_date? }      // message / nearest_date only when nothing was found
```

Open `client/src/modules/ai/RecommendPage.jsx` and **replace everything** with:

client/src/modules/ai/RecommendPage.jsx — 260 lines · complete file

```jsx
// Owner: Hooria. Route: /ai/recommend (registered in src/routes.jsx - do not rename this file or its
export).
// AI Recommendation (FR-RC-01, FR-RC-04 to FR-RC-06). The user describes what they need; Misbah's
// POST /api/ai/recommend returns up to 5 free assets in rank order, each with a reason.
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { ArrowUpRight, CalendarSearch, Info, Loader2, MapPin, SearchX, Wand2 } from 'lucide-react';
import api from '../../api/client';
import { useAuth } from '../../context/AuthContext';
import {
  Badge,
  Button,
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardIcon,
  CardTitle,
  DotPill,
  EmptyState,
  ErrorBox,
  Input,
  Label,
  Page,
  Skeleton,
  Textarea,
} from '../../components/ui';
import { fmtDate, plural, todayISO } from '../reports/format';
import { AssetThumb } from '../assets/parts';

const MAX_REQUIREMENT = 500; // FR-RC-01
const MAX_PURPOSE = 255;

export default function RecommendPage() {
  const { user } = useAuth();
  const isMember = user.role === 'Member';
  const [form, setForm] = useState({
    requirement: '',
    purpose: '',
    start_date: todayISO(),
    duration_days: 1,
  });
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const set = (field) => (e) => setForm((f) => ({ ...f, [field]: e.target.value }));

  async function search(values = form) {
    setLoading(true);
    setError(null);
    setResult(null);
    try {
```

```
      setResult(
        await api.post('/ai/recommend', { ...values, duration_days: Number(values.duration_days) }),
      );
    } catch (err) {
      setError(err);
    } finally {
      setLoading(false);
    }
  }

  function onSubmit(e) {
    e.preventDefault();
    search();
  }

  // FR-RC-06: try again from the nearest date the server suggested.
  function searchFrom(date) {
    const next = { ...form, start_date: date };
    setForm(next);
    search(next);
  }

  return (
    <Page
      title="AI"
      accent="recommendation"
      description="Describe what you need and when. The AI picks the best free assets and says why."
    >
      <div className="grid items-start gap-4 lg:grid-cols-[24rem_1fr]">
        <Card>
          <CardHeader>
            <CardTitle>
              <CardIcon icon={Wand2} /> What do you need?
            </CardTitle>
            <CardDescription>
              For example: "a laptop for video editing" or "a projector for a 20-person workshop".
            </CardDescription>
          </CardHeader>
          <CardContent>
            <form onSubmit={onSubmit} className="space-y-4">
              <div className="space-y-2">
                <Label htmlFor="requirement">Requirement</Label>
                <Textarea
                  id="requirement"
                  rows={4}
                  maxLength={MAX_REQUIREMENT}
                  value={form.requirement}
                  onChange={set('requirement')}
                  placeholder="Laptop for video editing"
                  required
                />
                <p className="text-muted-foreground text-right text-xs tabular-nums">
                  {form.requirement.length} / {MAX_REQUIREMENT}
                </p>
              </div>
              <div className="space-y-2">
                <Label htmlFor="purpose">Purpose</Label>
                <Input
                  id="purpose"
                  maxLength={MAX_PURPOSE}
                  value={form.purpose}
                  onChange={set('purpose')}
                  placeholder="Client video project"
                  required
                />
              </div>
              <div className="grid grid-cols-2 gap-3">
                <div className="space-y-2">
                  <Label htmlFor="start_date">Start date</Label>
                  <Input
                    id="start_date"
                    type="date"
                    min={todayISO()}
```

```
                value={form.start_date}
                onChange={set('start_date')}
                required
              />
            </div>
            <div className="space-y-2">
              <Label htmlFor="duration_days">Days</Label>
              <Input
                id="duration_days"
                type="number"
                min={1}
                max={30}
                value={form.duration_days}
                onChange={set('duration_days')}
                required
              />
            </div>
          </div>
          <Button
            type="submit"
            variant="brand"
            className="w-full"
            disabled={loading || !form.requirement.trim()}
          >
            {loading ? <Loader2 className="animate-spin" /> : <Wand2 />}
            {loading ? 'Finding the best match...' : 'Recommend assets'}
          </Button>
        </form>
      </CardContent>
    </Card>

    <div className="space-y-4">
      <ErrorBox error={error} />

      {loading && (
        <>
          <p className="text-muted-foreground flex items-center gap-2 text-sm">
            <Loader2 className="size-4 animate-spin" /> The AI is comparing free assets. This
            can take up to 10 seconds.
          </p>
          {[1, 2, 3].map((i) => (
            <Skeleton key={i} className="h-32 w-full rounded-3xl" />
          ))}
        </>
      )}

      {!loading && !result && !error && (
        <Card>
          <EmptyState icon={Wand2} title="Your recommendations will appear here">
            Fill in the form and press Recommend assets.
          </EmptyState>
        </Card>
      )}

      {result && (
        <>
          <div className="flex flex-wrap items-center gap-2 text-sm">
            <DotPill tone="brand">
              {fmtDate(result.start_date)} to {fmtDate(result.end_date)}
            </DotPill>
            <DotPill tone="neutral">{plural(result.duration_days, 'day')}</DotPill>
            {result.source === 'rules' && (
              <span className="text-muted-foreground flex items-center gap-1 text-xs">
                <Info className="size-3.5" /> Ranked by keyword match (the AI was not used)
              </span>
            )}
          </div>

          {result.recommendations.length === 0 ? (
            <Card>
              <EmptyState
                icon={SearchX}
                title="Nothing suitable is free"
```

```
                    action={
                      result.nearest_date && (
                        <Button variant="outline" onClick={() => searchFrom(result.nearest_date)}>
                          <CalendarSearch /> Search from {fmtDate(result.nearest_date)}
                        </Button>
                      )
                    }
                  >
                    {result.message || 'No free asset matches these dates.'}
                  </EmptyState>
                </Card>
              ) : (
                result.recommendations.map(({ rank, reason, asset }) => (
                  <Card key={asset.asset_id} className="flex-row items-stretch gap-4 p-4">
                    <div className="relative w-28 shrink-0 sm:w-40">
                      <AssetThumb asset={asset} className="h-full" />
                      <span
                        className={`absolute top-2 left-2 flex size-7 items-center justify-center rounded-
full text-xs font-bold ${rank <= 3 ? 'bg-highlight text-highlight-foreground' : 'bg-secondary'}`}
                      >
                        {rank}
                      </span>
                    </div>
                    <div className="flex min-w-0 flex-1 flex-col gap-2">
                      <div>
                        <Link
                          to={`/assets/${asset.asset_id}`}
                          className="font-display text-lg font-medium hover:underline"
                        >
                          {asset.name}
                        </Link>
                        <p className="text-muted-foreground flex flex-wrap items-center gap-x-2 text-xs">
                          <span>{asset.category}</span>
                          <span className="flex items-center gap-1">
                            <MapPin className="size-3" />
                            {asset.location}
                          </span>
                          <Badge variant="pill">{asset.condition}</Badge>
                        </p>
                      </div>
                      <p className="text-sm">{reason}</p>
                      {isMember && (
                        <div className="mt-auto">
                          <Button asChild size="sm">
                            <Link
                              to={`/request/${asset.asset_id}?
start=${result.start_date}&days=${result.duration_days}`}
                            >
                              Request <ArrowUpRight />
                            </Link>
                          </Button>
                        </div>
                      )}
                    </div>
                  </Card>
                ))
              )}
            </>
          )}
        </div>
      </div>
    </Page>
  );
}
```

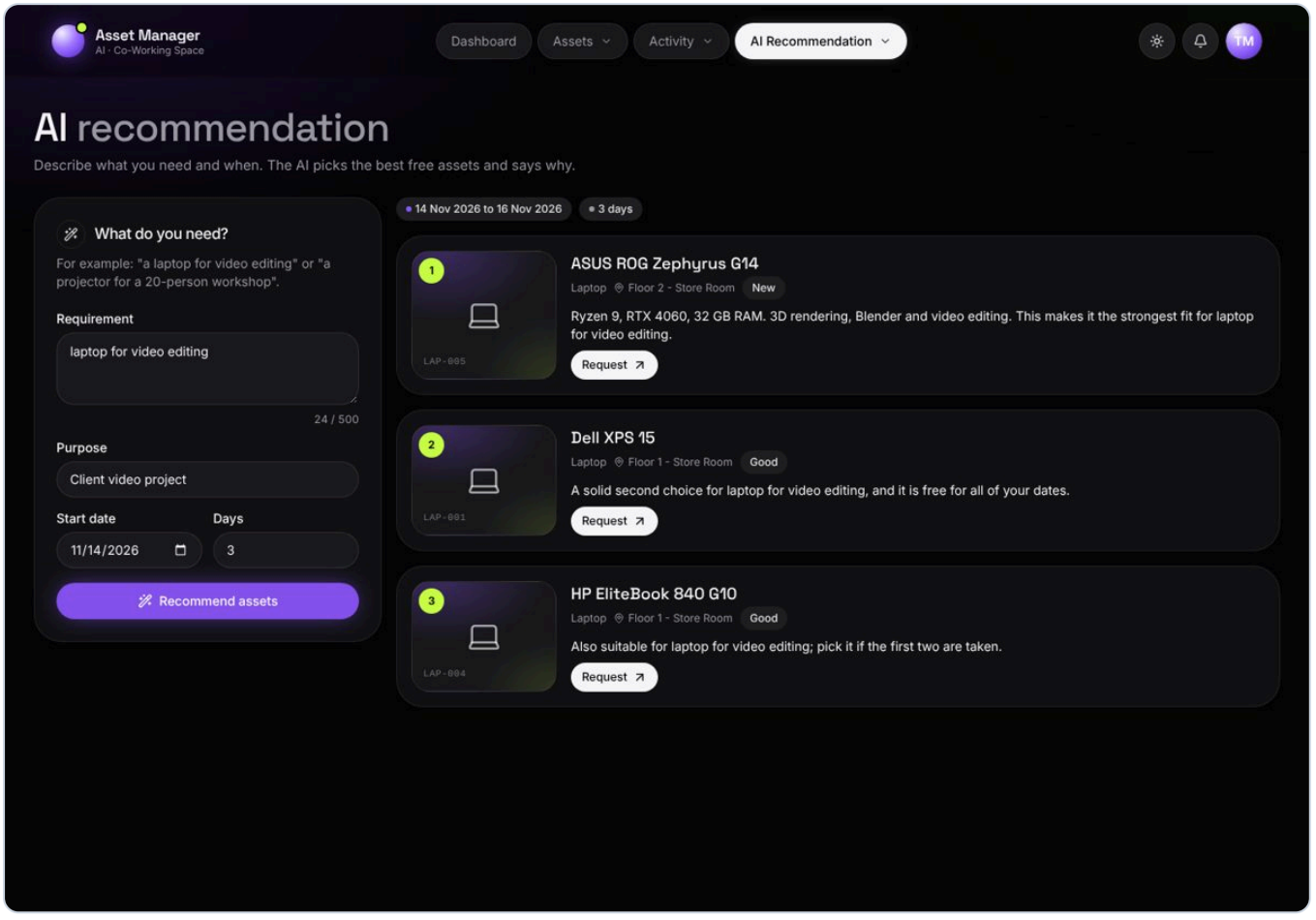
What the code does

- **The form (FR-RC-01)** is one `form` state object. `set('purpose')` returns an `onChange` handler for that field. The date input has `min={todayISO()}` so past dates can't be picked, days are 1 to 30, and the requirement has a live "24 / 500" counter.

- **Loading:** the AI can take up to 10 seconds, so the button shows a spinner and "Finding the best match...", and grey skeleton cards appear where the results will be.
- **Results (FR-RC-04):** up to 5 cards in rank order with picture, name, category, location, condition and the AI's reason. Ranks 1 to 3 get the lime badge. If `source` is `"rules"`, a small note says the keyword ranking was used.
- **Request button (FR-RC-05):** links to Hamza's form as `/request/<id>?start=<date>&days=<n>`; his form reads those two values and fills them in. Admins can't request assets, so `isMember` hides the button for them.
- **Nothing found (FR-RC-06):** shows the server's `message`, and when `nearest_date` is set a "Search from ..." button runs the same search again from that date.

Test it

1. As the member, open **AI → AI Recommendation**. Type "laptop for video editing", a purpose, a start date and 3 days, then press **Recommend assets**.
2. Press **Request** on the first result: Hamza's request form opens with your start date and 3 days already filled in.
3. Without an API key you see "Ranked by keyword match". Type something that doesn't exist (e.g. "zzqx"): you see the "Nothing suitable is free" message.
4. As the admin, run a search: there is no Request button.



AI Recommendation: three ranked laptops for "laptop for video editing", each with a reason and a Request button.

9 AI Assistant chat screen FR-AI-01, FR-AI-07

```
// You send: the new question + the earlier messages (oldest first, text only)
POST /api/ai/chat   { question: "Is any projector free tomorrow?", history: [ { role: "user",
content: "..." }, { role: "assistant", content: "..." } ] }
// You get back: the answer + buttons to show under it (may be an empty list)
{ answer: "Yes. Anker Nebula Capsule 3 is available...", links: [ { label: "View Anker Nebula Capsule
3", href: "/assets/14" } ] }
// Errors: 503 AI unavailable, 429 daily limit (50), 400 bad question. Show the "error" text as it
is.
GET /api/ai/usage   // { used: 6, limit: 50, remaining: 44, ai_available: true }
```

Open `client/src/modules/ai/AssistantPage.jsx` and **replace everything** with:

client/src/modules/ai/AssistantPage.jsx — 234 lines · complete file

```jsx
// Owner: Hooria. Route: /ai/assistant (registered in src/routes.jsx - do not rename this file or its
export).
// AI Assistant chat (FR-AI-01, FR-AI-07). The user asks questions in plain words; Misbah's
// POST /api/ai/chat answers using live data. Errors (AI off, daily limit) appear inside the chat.
import { useEffect, useRef, useState } from 'react';
import { Link } from 'react-router-dom';
import {
  AlertCircle,
  ArrowUpRight,
  Loader2,
  RotateCcw,
  SendHorizontal,
  Sparkles,
} from 'lucide-react';
import api from '../../api/client';
import {
  Button,
  Card,
  CardContent,
  CardHeader,
  CardIcon,
  CardTitle,
  DotPill,
  Page,
  Progress,
  Textarea,
} from '../../components/ui';
import { cn } from '@/lib/utils';

const MAX_QUESTION = 500; // FR-AI-01
const EXAMPLES = [
  'Is any projector free tomorrow?',
  'Which laptops are on floor 2?',
  'What are my requests?',
];

// One chat bubble. The user is on the right, the assistant on the left.
function Bubble({ msg }) {
  const mine = msg.role === 'user';
  return (
    <div className={cn('flex', mine ? 'justify-end' : 'justify-start')}>
      <div
        className={cn(
          'max-w-[85%] rounded-3xl px-4 py-3 text-sm whitespace-pre-wrap',
          mine && 'bg-primary text-primary-foreground rounded-br-md',
          !mine && !msg.error && 'bg-secondary rounded-bl-md',
          msg.error && 'border-danger/40 bg-danger/14 text-danger rounded-bl-md border',
        )}
      >
        {msg.error && <AlertCircle className="mr-1.5 mb-0.5 inline size-4" />}
        {msg.content}
        {msg.links?.length > 0 && (
          <div className="mt-3 flex flex-wrap gap-2">
            {msg.links.map((l) => (
```

```
              <Button key={l.href} asChild size="sm" variant="outline">
                <Link to={l.href}>
                  {l.label} <ArrowUpRight />
                </Link>
              </Button>
            ))}
          </div>
        )}
      </div>
    </div>
  );
}

export default function AssistantPage() {
  const [messages, setMessages] = useState([]); // [{ role: 'user' | 'assistant', content, links?, error? }]
  const [question, setQuestion] = useState('');
  const [sending, setSending] = useState(false);
  const [usage, setUsage] = useState(null);
  const bottomRef = useRef(null);

  const loadUsage = () =>
    api
      .get('/ai/usage')
      .then(setUsage)
      .catch(() => {});
  useEffect(() => {
    loadUsage();
  }, []);

  // Keep the newest message in view.
  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
  }, [messages, sending]);

  async function send(text = question) {
    const q = text.trim();
    if (!q || sending) return;
    // History = earlier turns as plain text, oldest first. Error bubbles are not part of the conversation.
    const history = messages
      .filter((m) => !m.error)
      .map(({ role, content }) => ({ role, content }));
    setMessages((list) => [...list, { role: 'user', content: q }]);
    setQuestion('');
    setSending(true);
    try {
      const { answer, links } = await api.post('/ai/chat', { question: q, history });
      setMessages((list) => [...list, { role: 'assistant', content: answer, links }]);
    } catch (err) {
      // FR-AI-07: 503 (AI unavailable), 429 (daily limit) or a network error, shown as a chat message.
      // 'Failed to fetch' / 'Request failed (502)' mean the server itself did not answer.
      const offline = /^(Failed to fetch|Request failed)/.test(err.message);
      const text = offline
        ? 'Cannot reach the server right now. Please try again in a moment.'
        : err.message;
      setMessages((list) => [...list, { role: 'assistant', content: text, error: true }]);
    } finally {
      setSending(false);
      loadUsage();
    }
  }

  function onKeyDown(e) {
    // Enter sends, Shift+Enter makes a new line.
    if (e.key === 'Enter' && !e.shiftKey) {
      e.preventDefault();
      send();
    }
  }

  return (
    <Page
      title="AI"
      accent="assistant"
```

```
      description="Ask about assets, availability and your requests in plain words."
      actions={
        messages.length > 0 && (
          <Button variant="outline" onClick={() => setMessages([])} disabled={sending}>
            <RotateCcw /> New chat
          </Button>
        )
      }
    >
      <div className="grid items-start gap-4 lg:grid-cols-[1fr_18rem]">
        <Card className="min-h-[28rem] gap-0 py-0">
          <div
            className="flex max-h-[60vh] min-h-[22rem] flex-col gap-3 overflow-y-auto p-5"
            aria-live="polite"
          >
            {messages.length === 0 && (
              <div className="m-auto flex max-w-md flex-col items-center gap-3 text-center">
                <span className="bg-orb size-14 rounded-full" aria-hidden="true" />
                <p className="font-display text-xl">How can I help?</p>
                <p className="text-muted-foreground text-sm">
                  I can check what is free, where things are and how your requests are going. Try
                  one of these:
                </p>
                <div className="flex flex-wrap justify-center gap-2">
                  {EXAMPLES.map((ex) => (
                    <Button key={ex} size="sm" variant="secondary" onClick={() => send(ex)}>
                      {ex}
                    </Button>
                  ))}
                </div>
              </div>
            )}
            {messages.map((m, i) => (
              <Bubble key={i} msg={m} />
            ))}
            {sending && (
              <div className="text-muted-foreground flex items-center gap-2 text-sm">
                <Loader2 className="size-4 animate-spin" /> Thinking...
              </div>
            )}
            <div ref={bottomRef} />
          </div>

          <form
            onSubmit={(e) => {
              e.preventDefault();
              send();
            }}
            className="border-border flex items-end gap-2 border-t p-4"
          >
            <div className="flex-1">
              <Textarea
                rows={2}
                maxLength={MAX_QUESTION}
                value={question}
                onChange={(e) => setQuestion(e.target.value)}
                onKeyDown={onKeyDown}
                placeholder="Ask a question..."
                aria-label="Your question"
                className="resize-none"
              />
              <p className="text-muted-foreground mt-1 text-right text-xs tabular-nums">
                {question.length} / {MAX_QUESTION}
              </p>
            </div>
            <Button
              type="submit"
              variant="brand"
              size="icon"
              className="mb-6"
              disabled={sending || !question.trim()}
              aria-label="Send"
            >
```

```
              <SendHorizontal />
            </Button>
          </form>
        </Card>

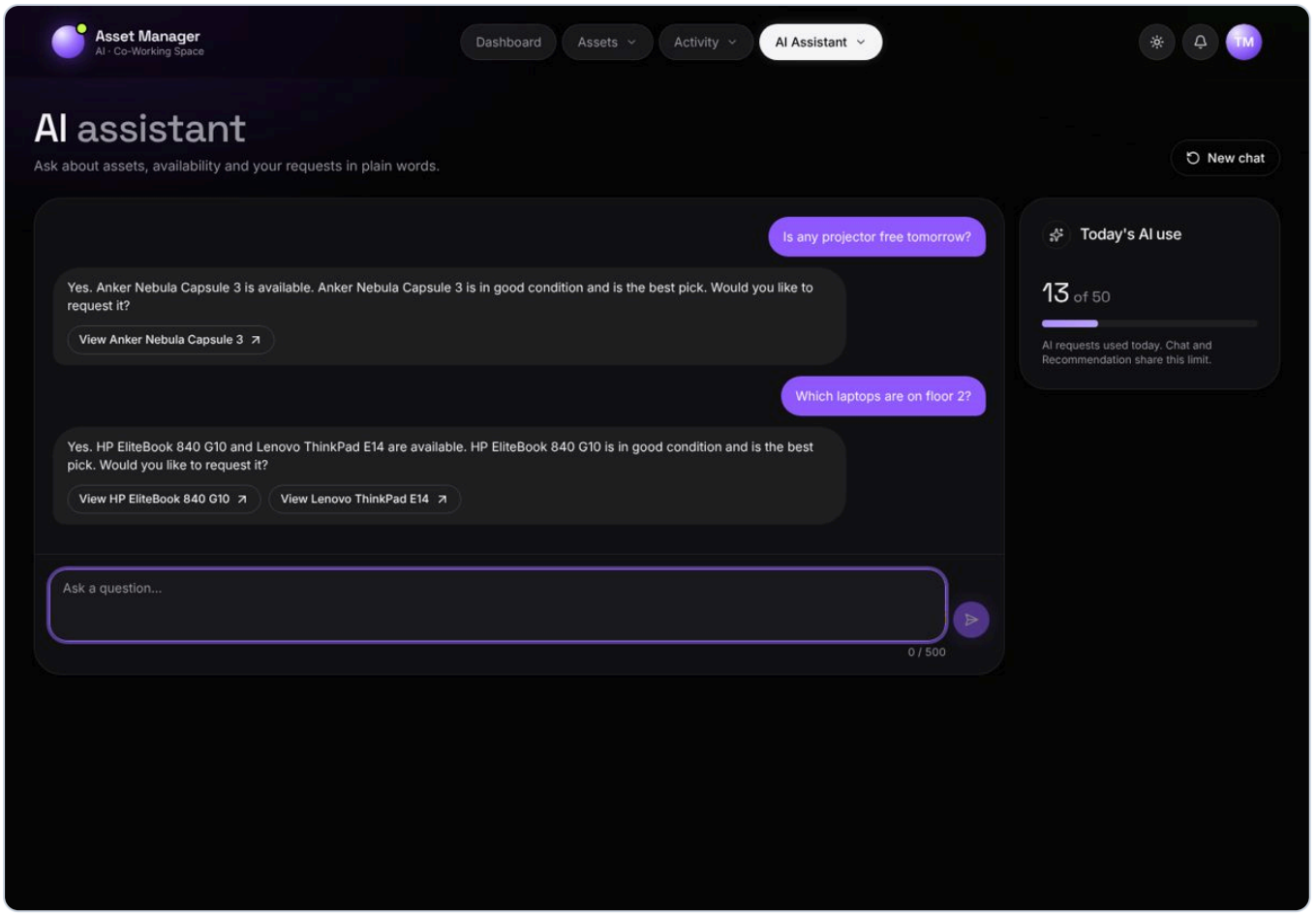
        <Card>
          <CardHeader>
            <CardTitle>
              <CardIcon icon={Sparkles} /> Today's AI use
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-3">
            {usage ? (
              <>
                <p className="font-display text-3xl">
                  {usage.used}
                  <span className="text-muted-foreground text-base"> of {usage.limit}</span>
                </p>
                <Progress value={(usage.used / usage.limit) * 100} />
                <p className="text-muted-foreground text-xs">
                  AI requests used today. Chat and Recommendation share this limit.
                </p>
                {!usage.ai_available && (
                  <DotPill tone="warning">AI is switched off on this server</DotPill>
                )}
              </>
            ) : (
              <p className="text-muted-foreground text-sm">Loading...</p>
            )}
          </CardContent>
        </Card>
      </div>
    </Page>
  );
}
```

What the code does

- `messages` is the whole conversation: `{ role, content, links?, error? }`. `Bubble` draws one message: the user's on the right in violet, the assistant's on the left in grey, errors in red.
- `send()` builds `history` from the earlier messages **before** adding the new one, leaves out error bubbles (they were never part of the conversation), shows the question at once, then waits for the answer.
- **Errors stay in the chat (FR-AI-07):** the `catch` turns any failure into a red assistant bubble with the server's own text. If the server can't be reached at all, it says so in plain words. The page never goes blank.
- **Keyboard:** Enter sends, Shift+Enter makes a new line. The box has `maxLength={500}` and a live counter (FR-AI-01).
- **Usage card:** "6 of 50" with a progress bar from `/ai/usage`, refreshed after every question. When the server has no API key, a note says the AI is switched off.
- **Example buttons** send a ready-made question with one click; `bottomRef` scrolls the newest message into view.

Test it

1. Open **AI → AI Assistant** and click "Is any projector free tomorrow?". You see "Thinking...", then the answer (or, without a key, the red "unavailable" message).
2. Ask a follow-up question: the assistant remembers the conversation. Buttons under an answer open that asset.
3. **Server stopped test:** stop the server (Ctrl+C in terminal 1) and send a question: a red "Cannot reach the server" bubble appears. Start it again and carry on.
4. Type more than 500 characters: the box stops at 500.



The AI Assistant with two answers, link buttons and the "6 of 50" usage card.

How you know you're done

Go through this list in the browser. All of these passed on the tested copy of your code.

- [ ] The bell shows the unread count, and opening a message or "Mark all as read" makes it drop at once.
- [ ] When the admin approves a member's request, that member's bell goes up within a minute.
- [ ] A loan due tomorrow gives its borrower exactly one reminder, even after restarting the server.
- [ ] My History shows the member's requests and borrowings, with no edit or delete buttons.
- [ ] The admin can open any asset's history from its details page; an unknown id shows "This asset does not exist".
- [ ] A member who types `/admin/history/1` is sent back to their dashboard.
- [ ] "Laptop for video editing, 3 days" shows up to 5 ranked results with reasons; **Request** opens Hamza's form with the dates filled in.
- [ ] Admins see recommendations but no Request button.
- [ ] The chat answers, remembers earlier questions, and shows link buttons.
- [ ] Without an API key the chat shows the "unavailable" message in red, inside the chat; stopping the server shows "Cannot reach the server".
- [ ] Every screen looks right in dark mode, light mode (sun button) and at phone width (390 px), with no sideways scrolling and no red errors in the browser console (F12).
- [ ] No placeholder card with your name is left anywhere in the app.

Save and share your work

```
git add server/src/modules/notifications server/src/modules/history client/src/modules/notifications
client/src/modules/history client/src/modules/ai
git commit -m "Hooria: notifications, usage history and AI screens"
git push -u origin hooria
```

Then open a Pull Request from `hooria` into `main` on GitHub and tell Shahan, who merges everyone's work.

If something goes wrong

You see	What it usually means
"Not implemented yet (owner: Hooria)"	The server is still running the old file. Save the file again or restart `npm run dev`.
401 "Please log in" in Postman	The `Authorization: Bearer <token>` header is missing, or the token is older than 24 hours. Log in again.
"Unknown column ..." in the server terminal	A typing mistake in the SQL. Compare the column names with `server/db/schema.sql`.
White page, red error in the console	Usually a wrong import name. Check the import lines at the top against this guide.
The bell never changes	Check `/api/notifications/unread-count` in Postman first; if it works, check the bell's `useEffect`.
Recommendation is slow, then shows keyword results	Normal: if the AI takes over 10 seconds, Misbah's server falls back to keyword ranking.

Where to look for more

Question	Where
Exact request and response shapes	`docs/API.md` ("Notes for the screens" is written for you)
How screens should look, every UI piece	`docs/UI_TEMPLATE.md`, or the live page at `/ui-template`
How the AI server works	`docs/modules/misbah.md`, or ask Misbah
What a requirement means	`docs/SRS.pdf` (search for the ID, e.g. FR-NT-01)
The short version of this guide	`docs/modules/Hooria_Task_Guide.pdf`